《低轨卫星网络中基于分段路由的软件定义组播》

Software Defined Multicast Using Segment Routing in LEO Satellite Networks

小组成员：蒋凯 万李裔 陈浙豫 程新越 任昶宇

目录

01 研究背景与网络模型

02 组播树构建与目标权衡

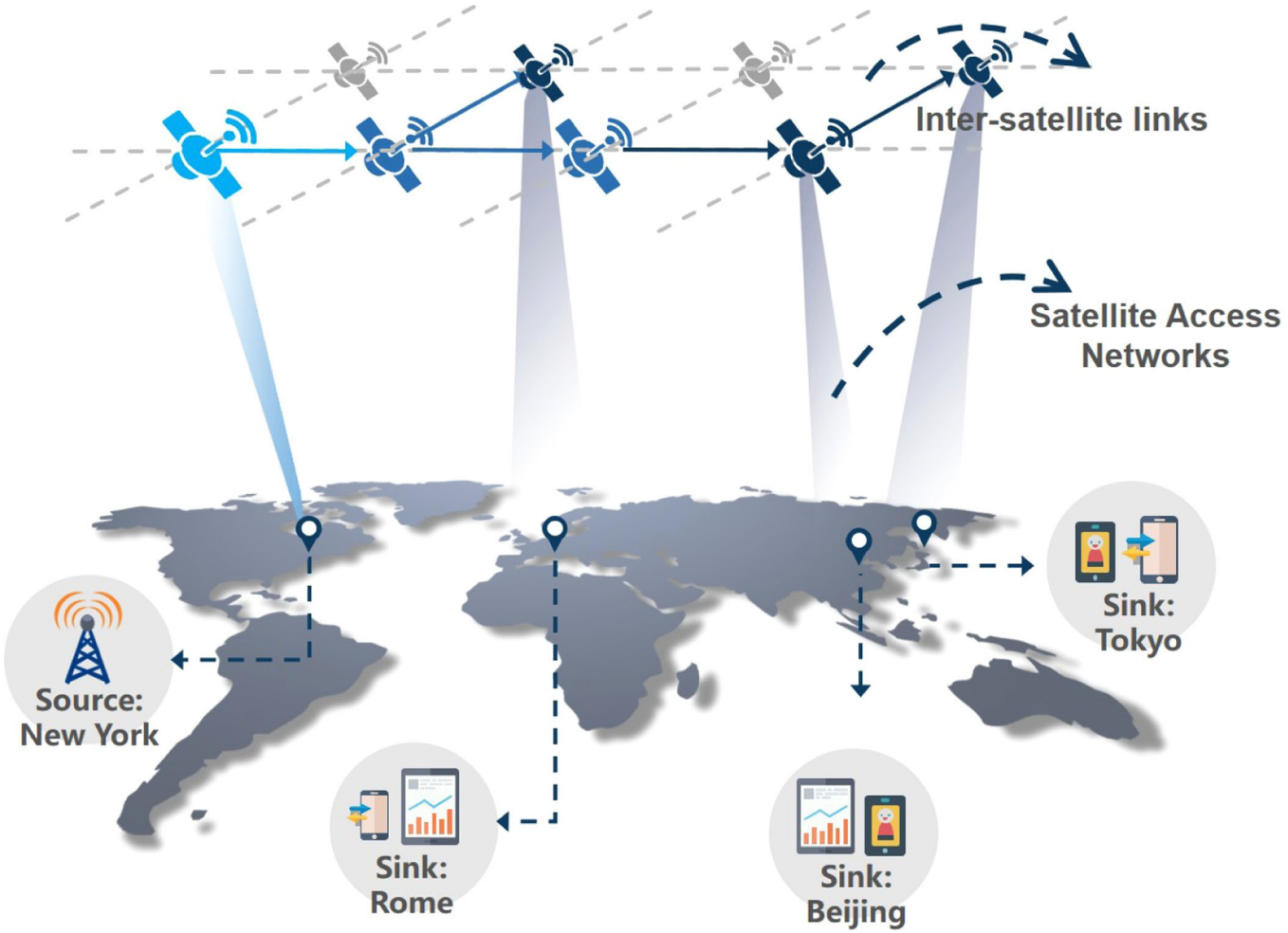
03 分段路由与转发机制

04 动态组管理与故障恢复

05 实验评估与主要结论

研究背景与网络模型

应用场景：通过 LEO 星座分发视频



共享链路上的一次传输

同一视频面向多个接收端。组播沿公共路径传送一份数据，在分支位置复制，再送达不同接收端。

两类网络中的带宽节省

LEO 星座可在卫星接入网（SAN）和星间链路（ISL）上执行组播，减少视频分发产生的重复流量。

集中控制的条件

同一机构部署的同构卫星星座可作为统一管理的网络，由 SDN 控制器掌握全局信息并制定路由策略。

研究动机：带宽、时延与可扩展性

带宽利用

传统 CDN 从分发节点到用户仍使用单播；传统 IP 组播采用分布式方式，难以利用全局信息优化组播树。

网络时延

仅缩短组播树总长度，不能保证每个源到接收端的路径最短。需要同时考虑总带宽消耗与端到端路径长度。

协议开销

星载设备的高速存储资源有限。组播状态与转发表规模会影响扩展能力，卫星切换和成员更新还会带来维护开销。

星座模型： Walker star 与四邻接链路

星座组成

采用 Walker star 型 LEO 星座： P 个轨道平面，每个平面有 Q 颗卫星，共 P×Q 颗同构卫星。

两类星间链路

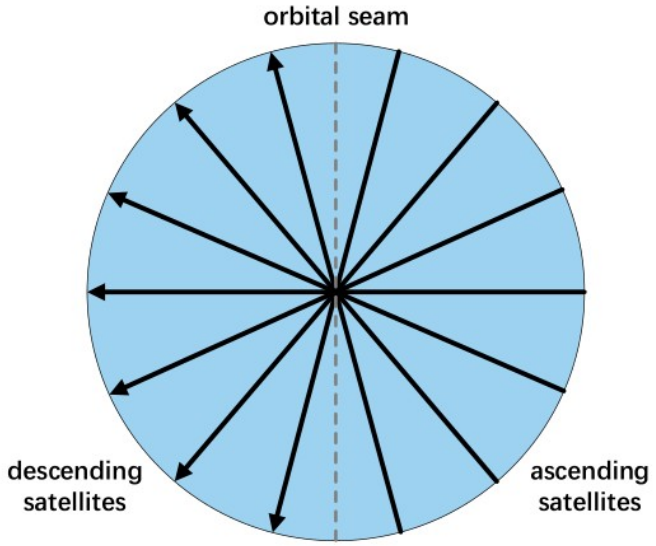
轨内 ISL 连接同一轨道中的相邻卫星；轨间 ISL 连接相邻轨道平面的卫星。

四邻接约束

每颗卫星与四颗相邻卫星通信：同轨道的上、下邻居，以及相邻轨道的左、右邻居。



轨道缝与极区：独立网格的形成



升轨与降轨

轨道缝将网络分为升轨与降轨两部分：一侧卫星由南向北运动，另一侧由北向南运动。

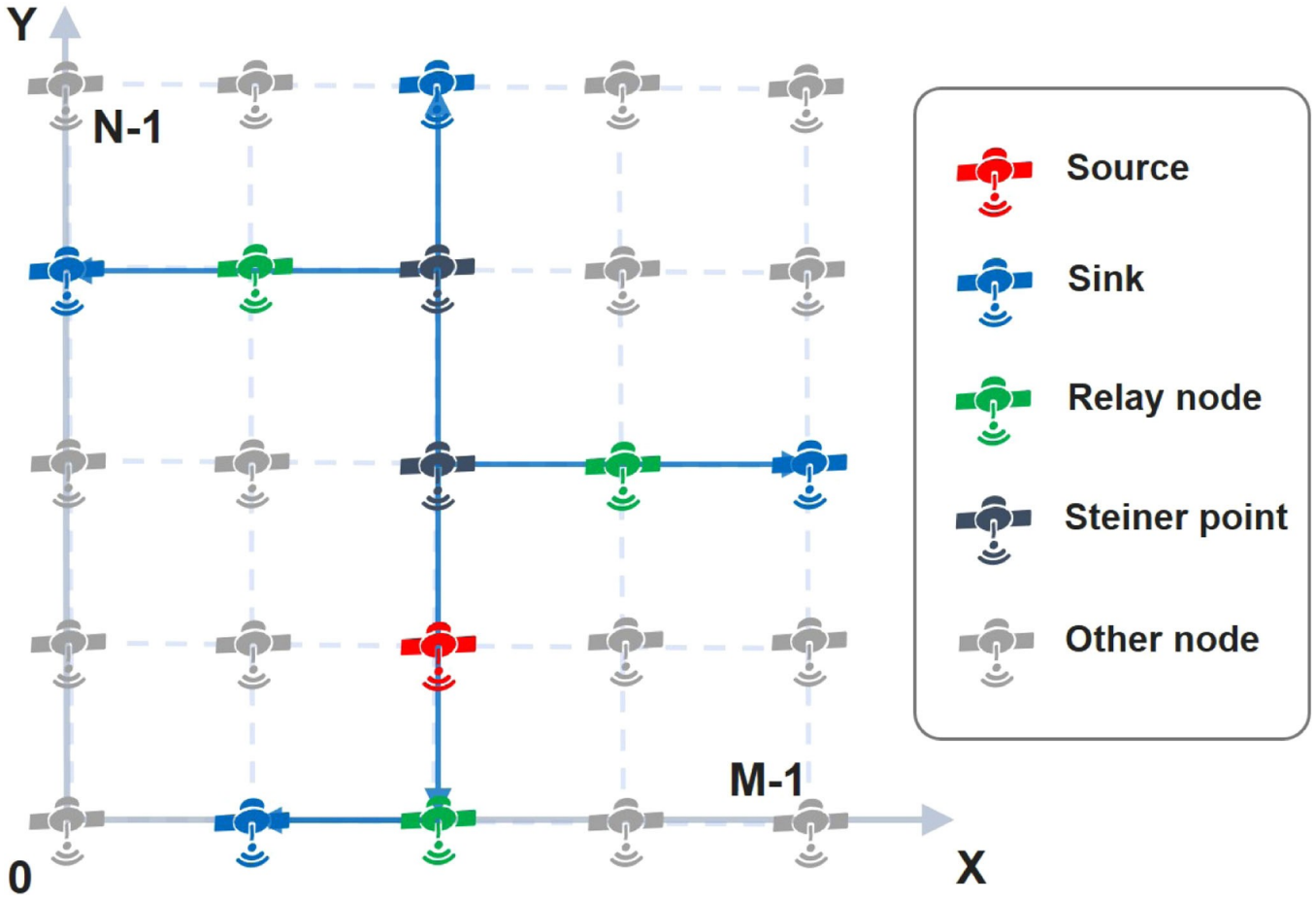
跨缝链路限制

轨道缝两侧卫星的相对速度较高，跨缝 ISL 无法建立；极区卫星休眠并关闭路由服务以节约能量。

网格建模范围

升轨与降轨分别形成独立网格。组播路由在一个网格内研究，两个网格之间的通信可经地面骨干网完成。

网格抽象：坐标、链路与控制器



图模型 G(V,E)

非极区的卫星构成 M×N 矩形网格。每颗卫星映射到二维逻辑坐标 (x,y)，星间链路对应图中的边。

距离与带宽假设

树构建采用直角距离，即横、纵方向跳数之和。模型中各条链路具有相同的带宽容量。

控制器的作用

地面 SDN 控制器已知源和接收端的坐标，计算组播树，建立和维护组播组，并执行路由策略。

时间片模型与整体技术路线

按时间片建模

将较长运行周期划分为短时隙。卫星轨道和速度已知，每个时隙的 ISL 拓扑可预先计算；用户连接的卫星在时隙内保持不变。

选树与转发

控制器依据组播目标选择 RST 或 RSA；将树编码为分段路由标签，通过逻辑地址定位节点，并在段内执行转发。

持续维护组播服务

逻辑地址适应卫星切换；成员加入、离开与网络故障触发树恢复；丢失数据通过局部根节点进行重传。

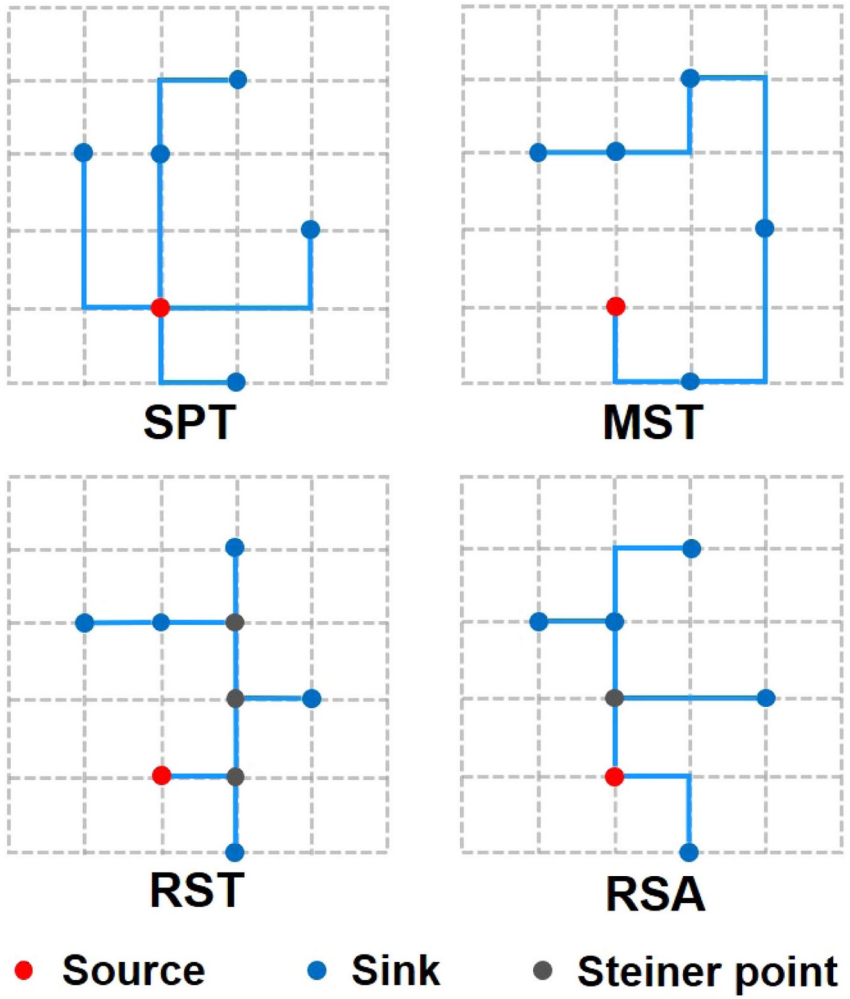
组播树构建与目标权衡

四类组播树：优化目标有什么不同

树类型	主要目标	源到接收端的路径
SPT： 最短路径树	保证各源—接收端路径最短	均为最短路径
MST： 最小生成树	最小化连接终端的边权总和	不保证最短
RST： 直角 Steiner 最小树	允许增加 Steiner 点，最小化直角树总长	不保证最短
RSA： 最小直角 Steiner 有向树	在根到各终端最短的约束下最小化树长	均为直角最短路径

Steiner 点是为连接终端而引入的额外节点，可使公共链路得到更充分的共享。

直观比较： SPT 、 MST 、 RST 与 RSA



相同终端，不同连接方式

红色为源，蓝色为接收端，灰色为 Steiner 点。四类树在相同网格和终端集合上形成不同的连接结构。

RST： 总树长优先

通过增加合适的 Steiner 点共享路径，减少组播使用的总链路跳数，从而降低总带宽消耗。

RSA： 兼顾最短到达路径

从源出发到每个接收端都保持直角最短路径，同时在这一约束下尽量缩短组播树。

RST： 以最小树长降低带宽消耗

问题定义

用水平、垂直线段连接全部终端，可加入额外的 Steiner 点，并使直角距离度量下的树总长度最小。网格内每跳对应一个直角距离单位。

采用的构建方法

采用基于生成图的 Zhou 启发式算法，以生成图构造初始树并筛选边替换候选。所选方法兼顾求解质量与速度，构建时间复杂度为 $O(n \log n)$ 。

确定转发方向

RST 构造过程本身不以源为起点生长。获得树后，将源指定为根，再沿树执行深度优先搜索，确定每个节点的父子关系。

RSA：在最短路径约束下节省带宽

问题定义

给定终端集合 T 和根 r，要求根到任意终端的树上路径都是直角最短路径；在满足该约束的树中，使总长度最小。

两项目标的结合

源到每个接收端的跳数保持最短，同时通过共享树上链路节省总带宽。因此 RSA 兼顾源—接收端时延与组播树总开销。

采用的构建方法

使用 minimal cover 算法：从源开始向外分轮覆盖接收端。该算法为 2- 近似算法，运行时间为 O(n log n)，其中 n 为组大小。

RST 与 RSA：构建方法比较

比较维度	RST	RSA
构建思路	借助生成图筛选边替换候选	从源开始分轮向外覆盖终端
采用算法	基于生成图的启发式算法	minimal cover 算法
构建复杂度	O(n log n)	O(n log n)
父子关系	构建后以源为根， DFS 确定	有向树以源为根
解法性质	启发式算法	2- 近似算法
路径约束	以总树长为目标，不保证各路径最短	根到每个终端均为直角最短路径

> n 表示组大小。 RST 以最小化树总长为目标； RSA 在根到各终端路径最短的约束下优化树总长。

组播树的选择依据

带宽节省为主

选择 RST。它以最小化总树长为目标，减少组播使用的总 ISL 跳数，适用于优先节省带宽的场景。

传播时延为主

选择 RSA。它保证源到各接收端的直角最短路径，在此约束下优化树长，兼顾最短到达路径与带宽节省。

与路由协议的衔接　RST 或 RSA 得到的树均可组织为有序的段列表。控制器据此生成标签栈，由源卫星封装到数据包头部，驱动后续组播转发。

分段路由与转发机制

逻辑地址：适应频繁的卫星切换

建立地面逻辑网格

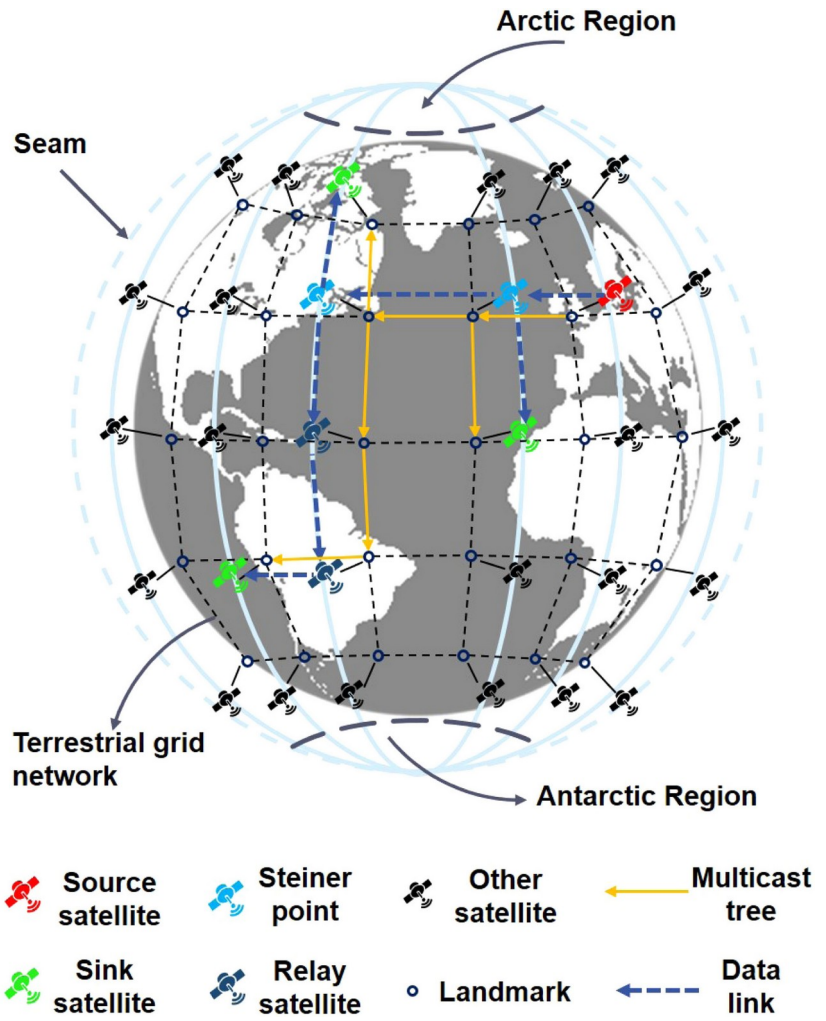
卫星垂直投影到地面地标，地标具有逻辑地址。地面被划分为以地标为中心的平行四边形区域。

按逻辑位置构建组播树

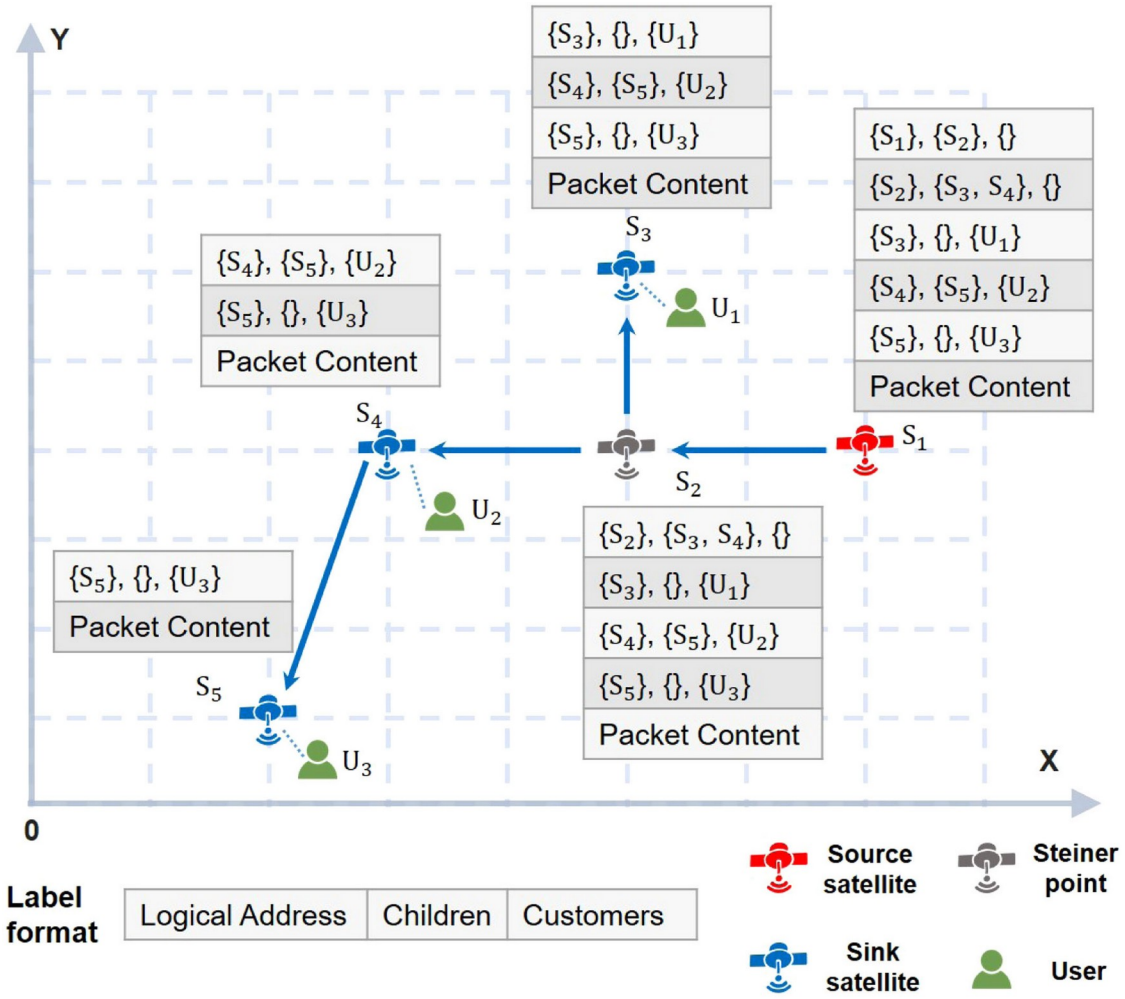
源、用户及当前服务卫星用关联地标表示。基于组成员的逻辑地址构造树，再映射到当前承载这些位置的卫星。

切换时更新映射

在卫星与用户时钟同步的条件下，卫星同步更新关联区域、自身及四邻居的逻辑地址；用户切换服务卫星后，仍沿用原逻辑地址。



源路由：把组播树写入标签栈



一个树边对应一个段

控制器计算组播树，将各段组织为有序列表。源卫星把列表编码为标签栈并加入数据包头部。

按广度优先顺序组织

从根开始按 BFS 组织标签，便于存储与搜索。栈顶为当前活动标签，指向该段的局部目的节点。

沿树推进活动标签

到达树成员节点后，按其子节点复制并更新数据包的活动标签，分别进入后续段。

标签结构与四类节点的处理职责

对象	字段或处理方式
标签：局部目的地址	记录当前段终点的逻辑地址
标签：子节点字段	记录树节点的子节点，用于启动后续段；可变长，含结束标记
标签：附属用户字段	记录当前卫星连接的用户地址；可变长，含结束标记
源卫星	按控制器提供的有序段列表，为数据包封装标签栈
中继卫星	根据活动标签朝局部目的地转发
Steiner / 接收端节点	按子节点复制并更新标签；向附属用户交付前移除报头

扩展的简表模式将段标签分散存入树成员卫星的转发表，包头仅保留一个标签。该模式减少报头开销，但会削弱敏捷性并增加控制平面复杂度。

段内 ECMP：等价最短路径转发

局部目的地决定方向

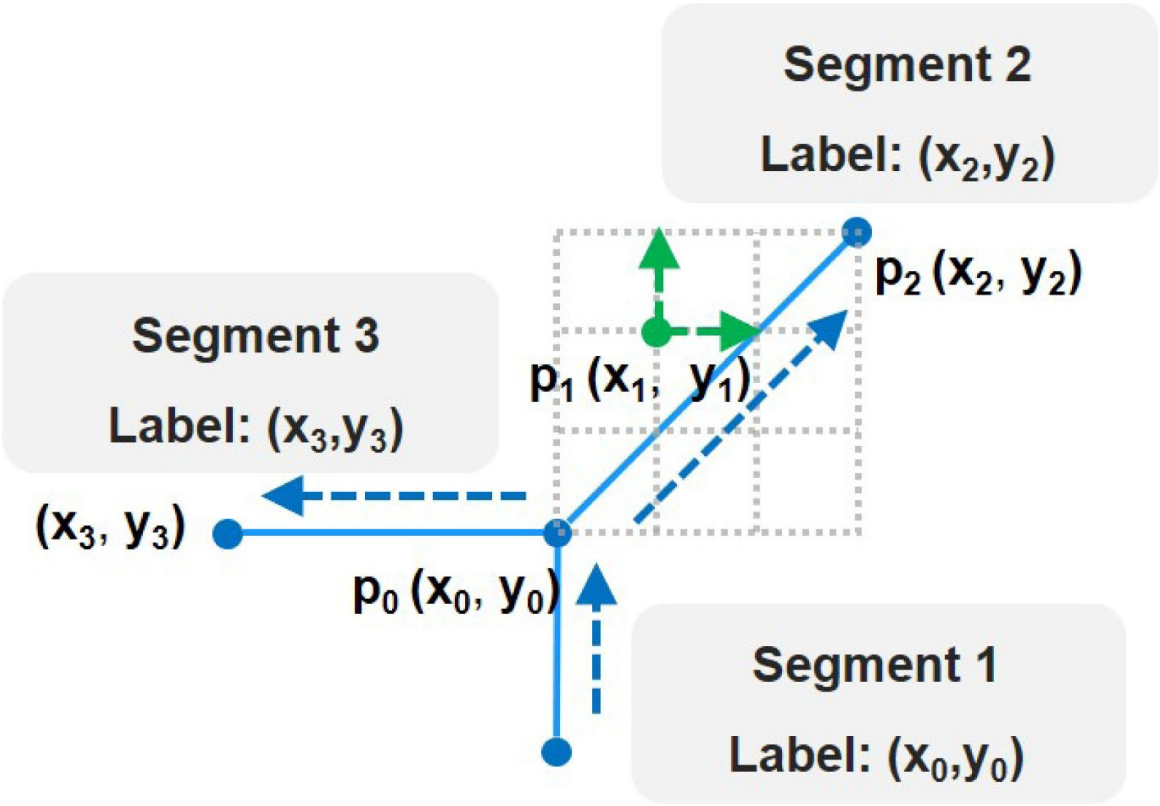
段内中继节点读取目的坐标。水平段或垂直段沿唯一的直线方向前进。

斜向段可随机分流

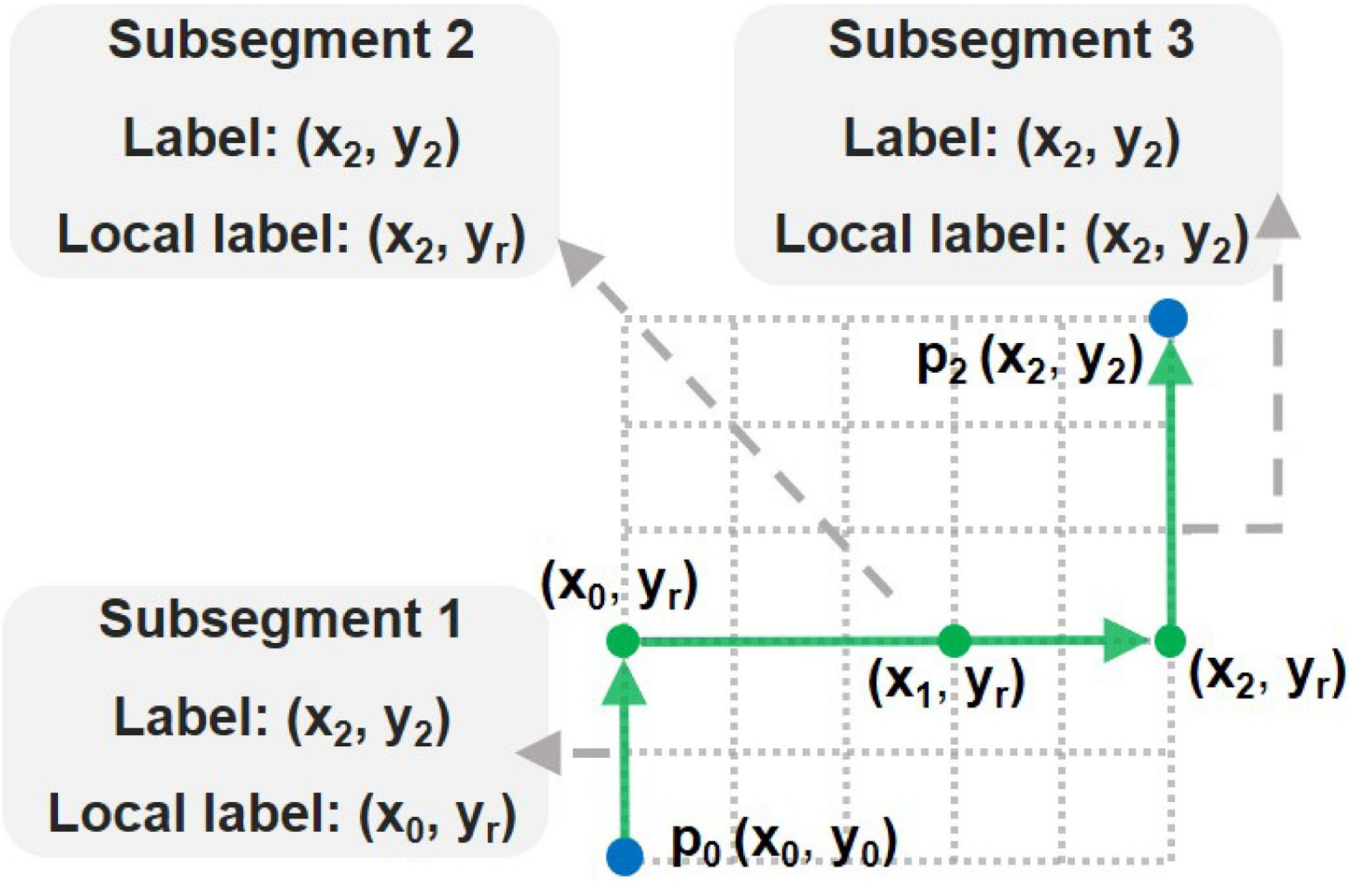
当横、纵坐标均未到达目标时，可随机选择朝目标前进的横向或纵向邻居；到达一条边界后沿另一方向继续。

缩小报头并实现负载分担

包头只需指定段终点，中间节点无需逐个记录。数据包可沿包围盒内多条等价最短路径到达目标。



局部 SR： 选择三子段路径



触发细粒度路径控制

控制器或卫星监测链路，根据阈值判断是否需要精细负载均衡；局部源将一个段进一步划分为子段。

用局部地标指定路径

在局部源到局部目的地之间选择一个地标，形成由三个水平或垂直子段组成的等价最短路径，并写入局部标签。

绕开故障候选

发生拥塞或故障时，排除经过受影响节点或链路的三子段路径，再从其余可用路径中选择。

ECMP 、局部 SR 与树管理的配合

普通段采用 ECMP

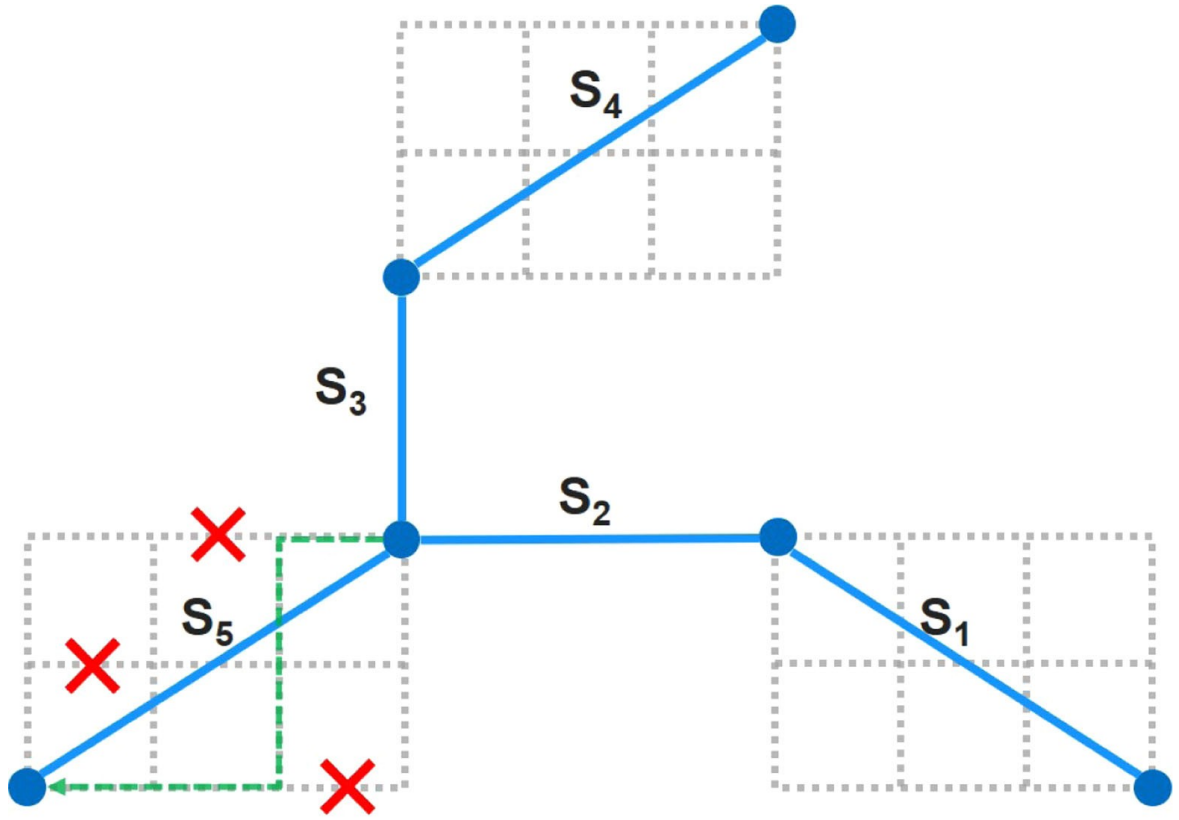
直线段及无特定约束的斜向段使用段内 ECMP ，减少路由记录，并提供较粗粒度的负载均衡。

受约束段采用局部 SR

具有细粒度负载均衡需求，或需要规避故障的斜向段，使用局部 SR 控制具体的三子段路径。

段内无法恢复时重构树

直线段发生故障，或故障使全部候选三子段路径失效时，进一步使用树管理机制处理。



动态组管理与故障恢复

成员加入与离开：统一为子树重连

成员加入

卫星把加入请求转交控制器。若组已注册，控制器执行加入操作；若组尚未注册，则建立新组并运行所选 RST 或 RSA 构建算法。

成员离开

控制器删除离开的终端、相邻 Steiner 点及相关边。操作产生的多个子树由树恢复机制重新连接。

统一处理形式

新加入的成员可视为仅有一个节点的子树。加入、离开及故障均可转化为重连若干子树的问题；恢复完成后更新源卫星中的路由规则。

树恢复示意： RST 边替换与 RSA 重连

RST： 连接点与边

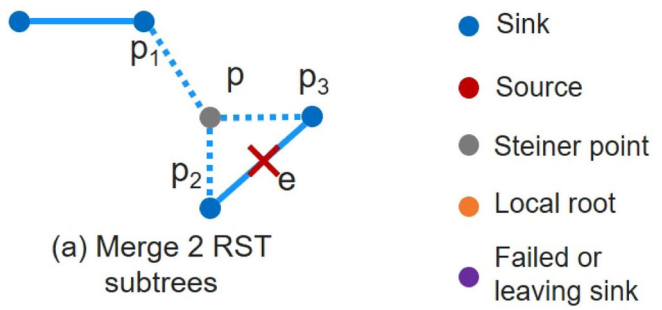


图 (a) 以 Steiner 点 p 连接 p_1 及边 e 的两个端点，再删除原边 e；新增连接的代价为 $d(p,p_1)$。

RST： 选择低代价候选

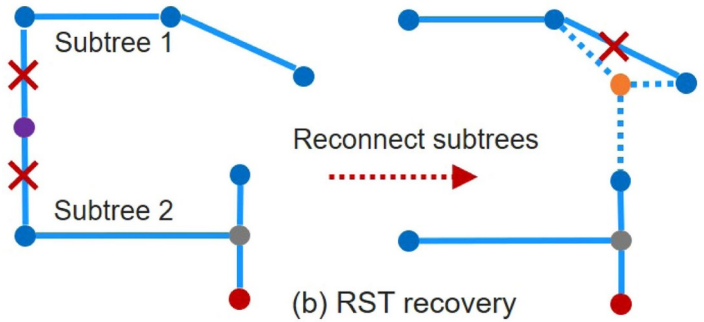


图 (b) 在不同子树间检查点—边对，选择代价最低的候选合并子树，迭代直至全部子树连通。

RSA： 连接局部根

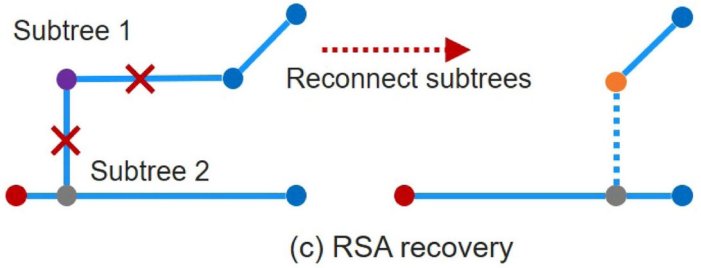


图 (c) 取各子树中距源最近的局部根，执行 minimal cover 算法，将这些根重新连接至源。

恢复操作：从候选连接到方向更新

RST 的候选筛选

成员变化或故障后先更新生成图。遍历子树边，并检查其他子树上的邻近点，寻找边替换代价最低的点—边对。

合并与父子关系更新

每轮选取当前最佳候选合并两个子树，可引入新的 Steiner 点，也可通过已有节点完成连接。全部重连后，从局部根开始更新父子关系。

RSA 的局部根处理

每个子树以距源最近的节点为局部根，仅对局部根执行 minimal cover 。 RST 与 RSA 所用恢复方法的时间复杂度均为 $O(\log n)$ 。

故障处理：恢复组播树并定位丢包子树

- **移除失效结构**

  检测到成员或链路失效后，控制器删除失效成员或边，以及相邻 Steiner 点和相关连接，触发树恢复。

- **避开受影响的候选**

  重连时排除经过失效节点或链路的连接候选。 RSA 使用的 minimal cover 方法可处理障碍，从而避免选用失效结构。

- **识别局部根**

  局部根是断开子树的根，用于定位需要重新接收数据的范围。若 RSA 中一个局部根通过另一个局部根连接至源，前者不再作为局部重传根。

局部重传：先到局部根，再覆盖子树

第一步：源到局部根

源卫星裁剪冗余标签栈，将丢失的数据包通过单播发送至需要重传的局部根。已经收到数据的其他节点不必重复接收。

第二步：局部子树组播

数据包从局部根沿恢复后的子树继续组播，只覆盖丢失数据的子树节点，减少全组重发带来的额外流量。

多个故障同时发生

可以用组播替代到多个局部根的单播，并建立临时树、增加相应路由信息。另一种处理方式是沿整棵树重新发送数据包。

不同变化由哪一层机制处理

变化或事件	对应机制	主要更新对象
卫星切换	地面地标与逻辑地址映射	卫星自身及邻居的逻辑地址
段内负载不均	ECMP 或局部 SR	下一跳选择或局部标签
局部可绕行的故障	排除受影响的三子段路径	段内路径
成员加入、离开	RST / RSA 树恢复	子树连接和源端路由规则
段内无法绕行的故障	删除失效结构并重连子树	组播树及相关标签栈
恢复后的丢包	局部根辅助的重传	重发范围和重传标签

> 不同机制分别作用于地址映射、段内路径、组播树和重传范围，共同适应动态卫星网络。

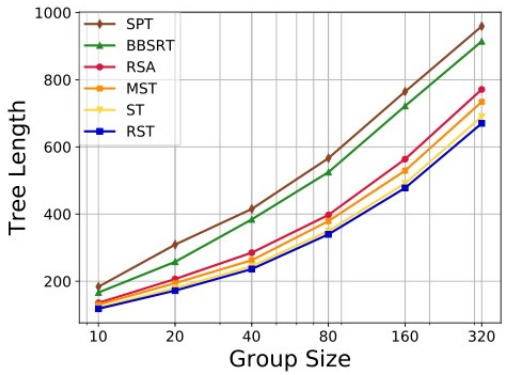
实验评估与主要结论

实验设置与五项评价指标

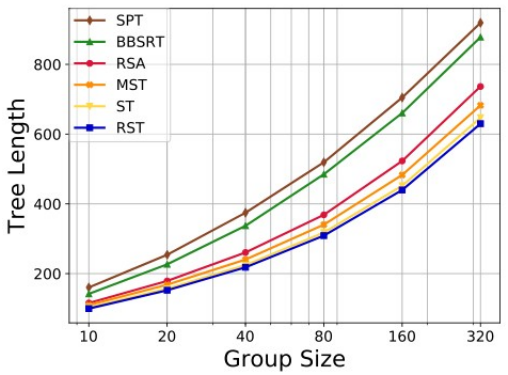
设置项	实验配置
网络规模	M×M 网格；默认 M=50，变化范围为 20–70
组大小	默认 80；主要取值为 10、20、40、80、160、320
组成员分布	均匀分布；非核心节点中 50% 或 100% 向核心节点聚集
对比算法	SPT、MST、ST、BBSRT、RST、RSA
重复次数	每组结果取 100 次试验的平均值

五项指标：树长、平均路径跳数、运行时间、简表模式表项缩减比例、ECMP 树长比例。树构建运行时间图还包含组大小为 640 的测试。

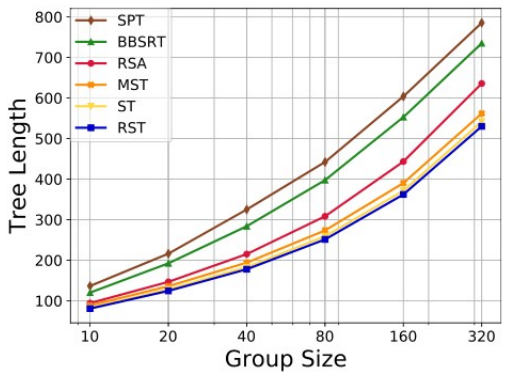
树长结果： RST 的带宽节省优势



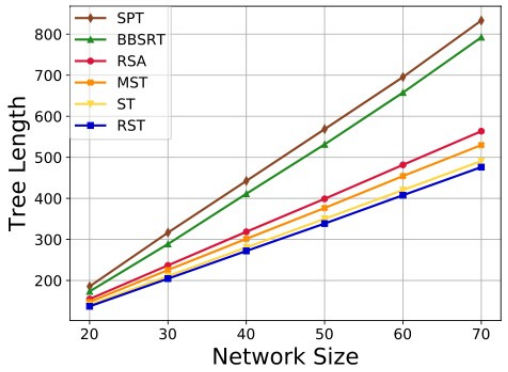
(a) Uniform distribution



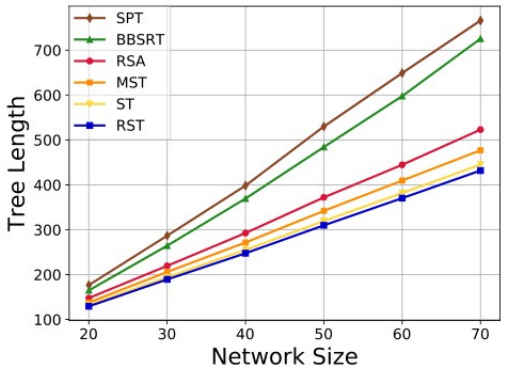
(b) 50% clustered non uniform



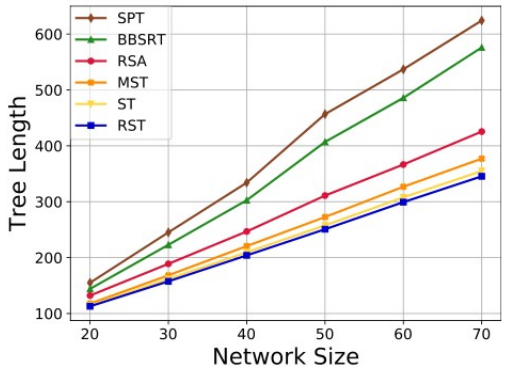
(c) 100% clustered non uniform



(d) Uniform distribution



(e) 50% clustered non uniform



(f) 100% clustered non uniform

总树长最短

三种成员分布、不同组大小与网络规模下，RST 均获得最短树长。

对比差距

相对 RST 的树长增幅：
ST：约 3%–5%
MST：平均约 10%
RSA：约 10%–20%
SPT / BBSRT：约 50%–100%。

成员聚集效应

组越大，共享链路的机会越多；成员越集中，树长通常越短。

平均时延结果： RSA 保持最短路径

按跳数衡量时延

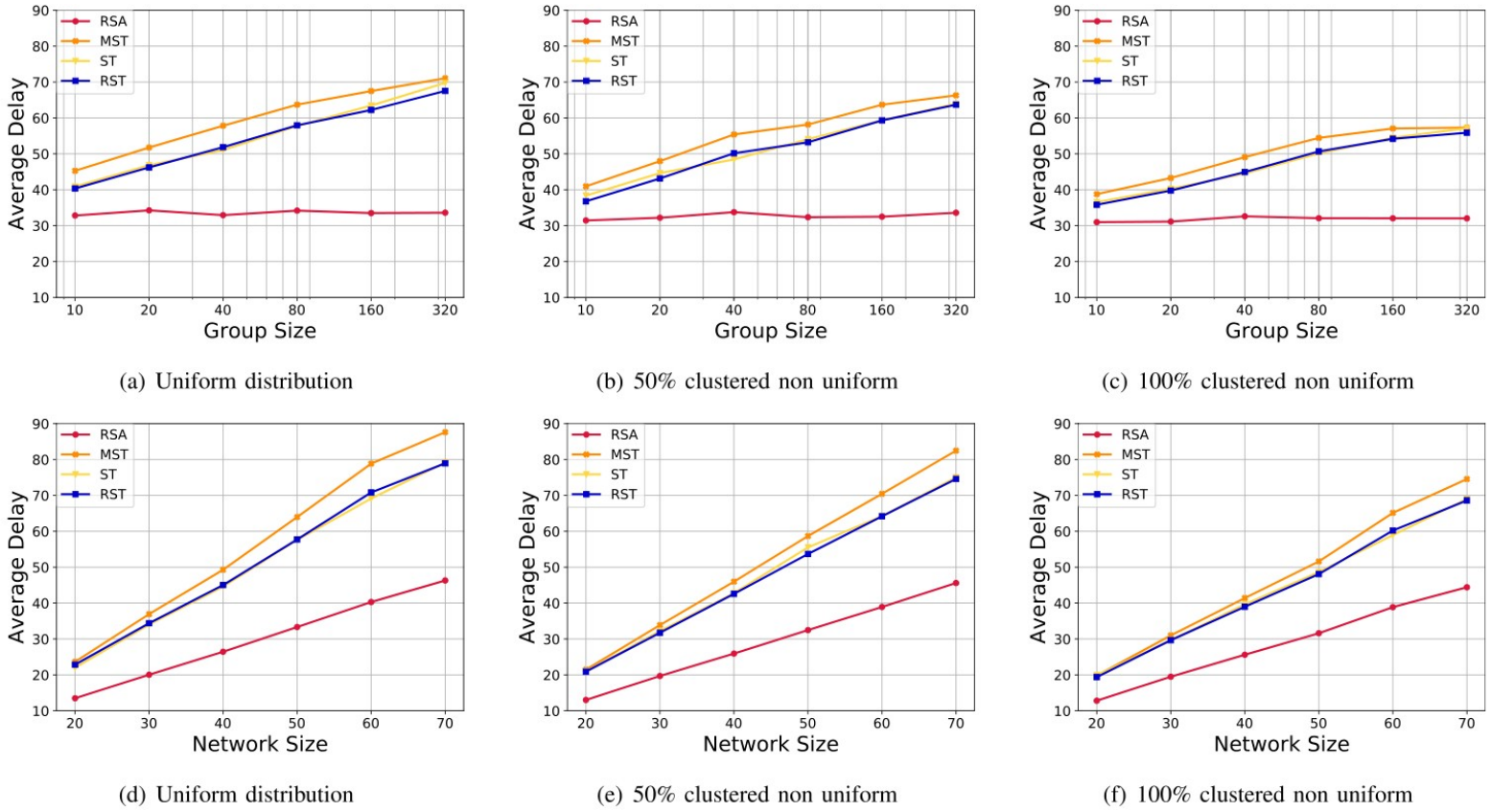
平均时延采用源到各接收端的树上平均路径跳数。 RSA 与 SPT 均保持最短路径。

组增大时的差异

RSA 的时延随组大小变化较稳定； RST 、 ST 与 MST 的时延明显上升。

RST 与 RSA 的权衡

RST 更侧重总树长； RSA 在保持最短到达路径的同时节省带宽，适合更关注传播时延的场景。

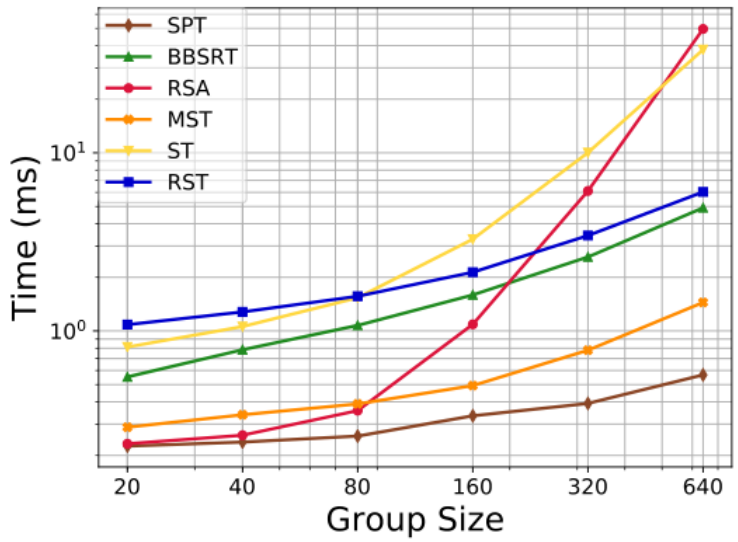


(a) Uniform distribution (b) 50% clustered non uniform (c) 100% clustered non uniform

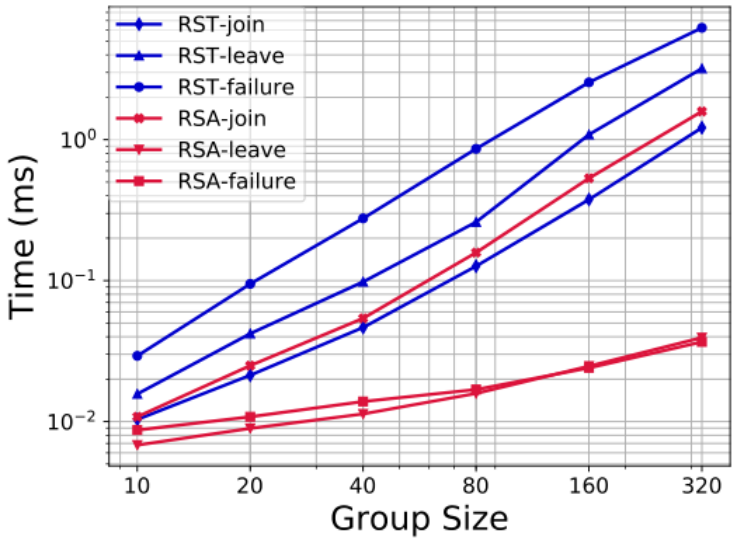
(d) Uniform distribution (e) 50% clustered non uniform (f) 100% clustered non uniform

运行时间：构建与恢复均可快速执行

树构建运行时间



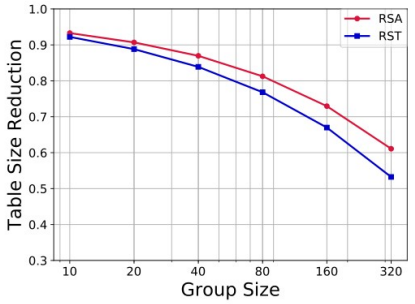
成员操作与故障恢复时间



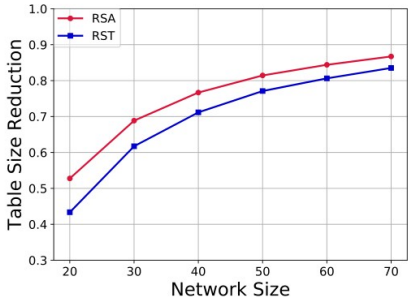
数百个组成员时，所选 RST / RSA 构建算法可在数毫秒内完成。组大小不超过 160 时，单次加入、离开或恢复操作约需 0.01–1 ms 。

SR 效果：路由表缩减与 ECMP 覆盖

简表模式：路由表项缩减比例

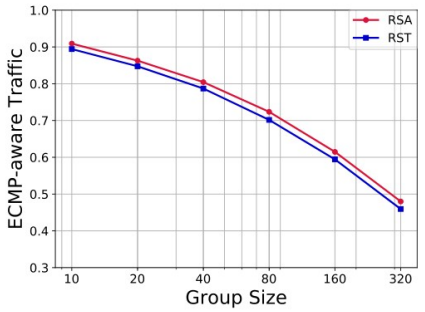


(a) group size

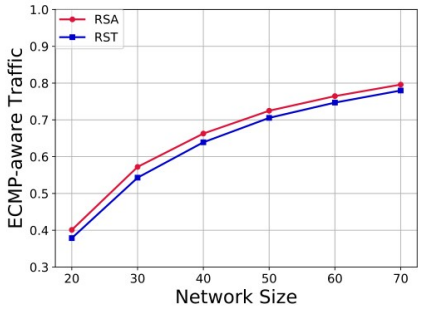


(b) network size

ECMP 覆盖比例（按树长统计）



(a) group size

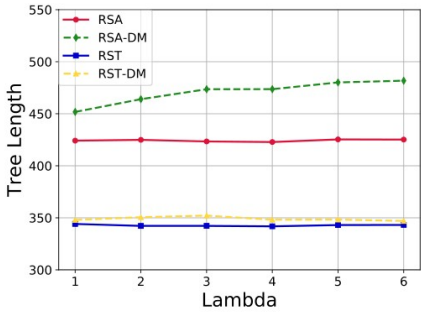


(b) network size

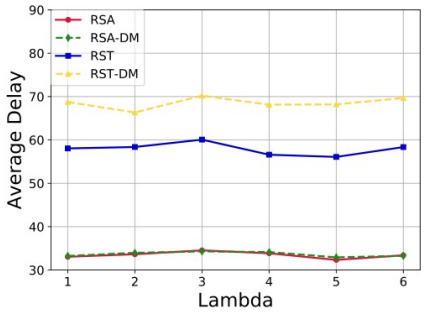
组小或网络大时，中继节点更多，表项节省更明显；斜向段更多， ECMP 覆盖更高。两项最高均约 90% 。 ECMP 比例＝采用 ECMP 的树长 / 总树长。

动态实验与结论：保持性能，按目标选树

动态成员： RST-DM / RSA-DM

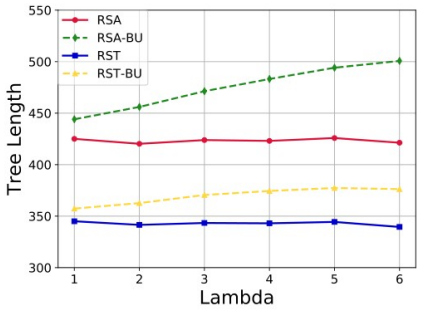


(a) Tree length

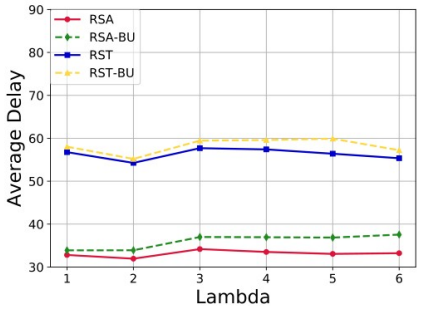


(b) Average delay

动态故障： RST-BU / RSA-BU



(a) Tree length



(b) Average delay

成员变化与故障按均值为 1/λ 的指数分布发生。动态管理下性能总体稳定； RSA 成员变化的时延恶化最高可达约 30% 。带宽优先选 RST ，传播时延优先选 RSA 。

感谢聆听

敬请老师批评指正